VRITARA

Smart Safety System

Backend API Documentation

Version: 1.0

Environment: Production

Base URL: https://e0538aec-ddf0-4112-ab9f-3752d0d16dab-00-317onqis7vczd.pike.replit.dev

Prepared For: PCB & Hardware Integration Vendor

Date: February 12, 2026

1. Project Overview

VRITARA Smart Safety System is a comprehensive personal safety platform designed for real-time emergency detection, alerting, and incident management. The system integrates a mobile/web application with IoT-enabled wearable hardware (smartwatch) to provide both manual and automatic SOS capabilities.

Core Features

- **Smart SOS System:** Toggle-based emergency state management with both manual and automatic trigger modes. The SOS state persists in the database and survives app restarts.
- **Manual SOS:** User-triggered emergency via app button. Creates incident, sends alerts to emergency contacts, and broadcasts to nearby users.
- **Automatic SOS:** Hardware-triggered emergency based on sensor data (sound level > 80dB, motion level > 1.5g). The smartwatch sends sensor readings to the backend which processes and triggers alerts automatically.
- **JWT Authentication:** Secure user authentication with JSON Web Tokens (24-hour expiry), bcrypt password hashing, OTP login simulation, and password reset flow.
- **Nearby Device Broadcast:** When an SOS is triggered, the system uses the Haversine formula to identify all VRITARA users within a 200-meter radius and sends them emergency broadcast alerts.
- **Emergency Contacts:** Users can register up to 3 emergency contacts. During SOS, simulated SMS alerts are sent to all contacts with the user's location.
- **Media Storage:** Automatic evidence capture during SOS (BMP images + WAV audio). Media files are linked to incidents and can be securely deleted with password verification.
- **Incident Logging System:** Complete incident lifecycle tracking with status management (active/resolved), SMS logs, nearby broadcast logs, and attached media records.

2. Authentication Flow

2.1 Signup Process

1. User sends POST request to `/api/signup` with username, email, phone, and password.
2. Server validates all fields are present and email is not already registered.
3. Password is hashed using **bcrypt** (10 salt rounds).
4. User record is created in PostgreSQL database.
5. Server returns a **JWT token** (valid for 24 hours) and user profile data.

2.2 Login Process

1. User sends POST request to `/api/login` with email and password.
2. Server looks up user by email and verifies password using **bcrypt.compare**.
3. On success, server returns a new JWT token and user profile data.

2.3 OTP Login (Simulated)

1. User requests OTP via `/api/request-otp` with their email.
2. Server generates a 6-digit OTP code, stores it with a 5-minute expiry.
3. OTP is returned in the response (simulated — in production, this would be sent via SMS/email).
4. User verifies OTP via `/api/verify-otp` to receive JWT token.

2.4 JWT Token Usage

All authenticated endpoints require the JWT token in the Authorization header:

```
Authorization: Bearer <JWT_TOKEN>
```

> **Token Expiry:** JWT tokens expire after 24 hours. After expiry, the user must login again to obtain a new token.

3. API Endpoints

All endpoints use the base URL: `https://e0538aec-ddf0-4112-ab9f-3752d0d16dab-00-317onqis7vczd.pike.replit.dev`

3.1 Server Status

GET `/status` **No Auth**

Health check endpoint to verify the API server is running and responsive.

Success Response (200):

```
{
  "status": "API is live"
}
```

3.2 Signup

POST `/api/signup` **No Auth**

Register a new user account. Returns JWT token on successful registration.

Request Headers:

Key	Value
Content-Type	application/json

Request Body:

```
{
  "username": "testuser",
  "email": "testuser@gmail.com",
  "phone": "9876543210",
  "password": "Test@123"
}
```

Success Response (201):

```
{
  "message": "User registered successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
  "user": {
    "id": 1,
    "username": "testuser",
    "email": "testuser@gmail.com",
    "phone": "9876543210"
  }
}
```

Error Responses:

Code	Condition	Response
400	Missing fields	{"error": "All fields are required"}
409	Email exists	{"error": "Email already registered"}
500	Server error	{"error": "Server error"}

3.3 Login

POST `/api/login` **No Auth**

Authenticate an existing user with email and password. Returns JWT token on success.

Request Headers:

Key	Value
Content-Type	application/json

Request Body:

```
{
  "email": "testuser@gmail.com",
  "password": "Test@123"
}
```

Success Response (200):

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
  "user": {
    "id": 1,
    "username": "testuser",
    "email": "testuser@gmail.com",
    "phone": "9876543210"
  }
}
```

Error Responses:

Code	Condition	Response
400	Missing fields	{"error": "Email and password are required"}
401	Wrong credentials	{"error": "Invalid credentials"}
500	Server error	{"error": "Server error"}

3.4 Request OTP

POST `/api/request-otp` **No Auth**

Request a 6-digit OTP code for passwordless login. OTP is valid for 5 minutes.

Request Body:

```
{ "email": "testuser@gmail.com" }
```

Success Response (200):

```
{
  "message": "OTP sent successfully (check server logs for simulation)",
  "otp": "482913"
}
```

Error Responses:

Code	Condition	Response
400	Missing email	{"error": "Email is required"}
404	Email not found	{"error": "Email not found"}

3.5 Verify OTP

POST `/api/verify-otp` **No Auth**

Verify OTP code and login. Returns JWT token on success.

Request Body:

```
{ "email": "testuser@gmail.com", "otp": "482913" }
```

Success Response (200):

```
{
  "message": "OTP verified, login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
  "user": { "id": 1, "username": "testuser", "email": "testuser@gmail.com", "phone": "9876543210" }
}
```

Error Responses:

Code	Condition	Response
400	Missing fields	{"error": "Email and OTP are required"}
401	Invalid/expired OTP	{"error": "Invalid or expired OTP"}

3.6 Forgot Password

POST `/api/forgot-password` **No Auth**

Generate a password reset token. Token is valid for 15 minutes.

Request Body:

```
{ "email": "testuser@gmail.com" }
```

Success Response (200):

```
{
  "message": "Password reset token generated (simulated)",
  "resetToken": "a1b2c3d4"
}
```

3.7 Reset Password

POST `/api/reset-password` **No Auth**

Reset user password using a valid reset token.

Request Body:

```
{
  "email": "testuser@gmail.com",
  "resetToken": "a1b2c3d4",
  "newPassword": "NewPass@456"
}
```

Success Response (200):

```
{ "message": "Password reset successful. You can now login with your new password." }
```

3.8 Get User Profile

GET `/api/user/profile` **Bearer Token**

Retrieve the authenticated user's profile information.

Success Response (200):

```
{
  "id": 1,
  "username": "testuser",
  "email": "testuser@gmail.com",
  "phone": "9876543210",
  "created_at": "2026-02-12T10:00:00.000Z"
}
```

3.9 Update User Profile

PUT `/api/user/profile` **Bearer Token**

Update the authenticated user's username and/or phone number.

Request Body:

```
{ "username": "newname", "phone": "9999999999" }
```

Success Response (200):

```
{
  "message": "Profile updated",
  "user": { "id": 1, "username": "newname", "email": "testuser@gmail.com", "phone": "9999999999" }
}
```

3.10 Toggle SOS (Primary SOS Endpoint)

POST `/api/sos/toggle` **Bearer Token**

Toggle the user's emergency state between **normal** and **emergency**. When activating: creates incident, sends SMS to contacts, broadcasts to nearby users. When deactivating: resolves the active incident. Uses database transactions for atomic state changes.

Request Body:

```
{
  "latitude": 12.9716,
  "longitude": 77.5946,
  "message": "Emergency SOS triggered"
}
```

Activation Response (201):

```
{
  "state": "emergency",
  "incident": {
    "id": 1,
    "user_id": 4,
    "type": "manual",
    "latitude": 12.9716,
    "longitude": 77.5946,
    "message": "Emergency SOS triggered",
    "status": "active",
    "created_at": "2026-02-12T10:30:00.000Z"
  },
  "notifications": {
    "sms_sent": 2,
    "sms_details": [
      { "contact": "Father", "phone": "9876543210", "status": "sent" }
    ],
    "nearby_broadcasts": 1,
    "nearby_details": [
      { "receiver": "nearby_user", "distance": "150m", "status": "sent" }
    ]
  }
}
```

Deactivation Response (200):

```
{
  "state": "normal",
  "resolved_incident_id": 1,
  "message": "Emergency resolved"
}
```

3.11 Get SOS State

GET `/api/sos/state` **Bearer Token**

Get the current emergency state of the authenticated user.

Success Response (200):

```
{
  "state": "normal",
  "active_incident_id": null
}
```

3.12 Trigger Manual SOS

POST `/api/sos/manual` **Bearer Token**

Trigger a manual SOS emergency. Creates an incident log, sends simulated SMS to all emergency contacts, broadcasts to nearby users within 200m, and links any provided media files to the incident.

Request Body:

```
{
  "latitude": 12.9716,
  "longitude": 77.5946,
  "message": "Manual emergency triggered",
  "media_ids": [1, 2]
}
```

Success Response (201):

```
{
  "emergency": true,
  "message": "SOS signal sent successfully",
  "incident": {
    "id": 5,
    "user_id": 4,
    "type": "manual",
    "latitude": 12.9716,
    "longitude": 77.5946,
    "status": "active",
    "created_at": "2026-02-12T10:35:00.000Z"
  },
  "notifications": {
    "sms_sent": 2,
    "sms_details": [...],
    "nearby_broadcasts": 0,
    "nearby_details": []
  },
  "media": [...]
}
```

3.13 Trigger Automatic SOS

POST `/api/sos/automatic` **Bearer Token**

Trigger an automatic SOS based on sensor data from the smartwatch/wearable device. The backend processes sound and motion levels and creates an incident with full notification flow. This is the primary endpoint for PCB/hardware integration.

Request Body:

```
{
  "latitude": 12.9716,
  "longitude": 77.5946,
  "sound_level": 85,
  "motion_level": 3.5,
  "media_ids": []
}
```

Success Response (201):

```
{
  "emergency": true,
  "message": "Automatic SOS triggered",
  "incident": {
    "id": 6,
    "user_id": 4,
    "type": "automatic",
    "latitude": 12.9716,
    "longitude": 77.5946,
    "message": "Auto-detected: sound=85dB, motion=3.5g",
    "sound_level": 85,
    "motion_level": 3.5,
    "status": "active",
    "created_at": "2026-02-12T10:40:00.000Z"
  },
  "notifications": {
    "sms_sent": 2,
    "sms_details": [...],
    "nearby_broadcasts": 0,
    "nearby_details": []
  },
  "media": []
}
```

3.14 Get Incident History

GET `/api/incidents` **Bearer Token**

Retrieve all incidents for the authenticated user, including SMS notifications, nearby broadcasts, and attached media files. Also available at `/api/sos/incidents`.

Success Response (200):

```
{
  "incidents": [
    {
      "id": 6,
      "user_id": 4,
      "type": "automatic",
      "latitude": 12.9716,
      "longitude": 77.5946,
      "message": "Auto-detected: sound=85dB, motion=3.5g",
      "sound_level": 85,
      "motion_level": 3.5,
      "status": "resolved",
      "created_at": "2026-02-12T10:40:00.000Z",
      "sms_notifications": [
        { "contact_name": "Father", "contact_phone": "9876543210", "status": "sent" }
      ],
      "nearby_broadcasts": [],
      "media_files": [
        { "id": 3, "filename": "1707...-img.bmp", "mimetype": "image/bmp", "file_size": 230454 }
      ]
    }
  ]
}
```

3.15 Update Location

POST `/api/location` **Bearer Token**

Update the user's current GPS location. This is used for location tracking and nearby user broadcast calculations.

Request Body:

```
{
  "latitude": 12.9716,
  "longitude": 77.5946
}
```

Success Response (200):

```
{
  "message": "Location updated",
  "location": {
    "id": 10,
    "user_id": 4,
    "latitude": 12.9716,
    "longitude": 77.5946,
    "created_at": "2026-02-12T10:45:00.000Z"
  }
}
```

3.16 Add Emergency Contact

POST `/api/contacts` **Bearer Token**

Add an emergency contact (maximum 3 per user). These contacts receive SMS alerts during SOS events.

Request Body:

```
{
  "name": "Father",
  "phone": "9876543210",
  "relationship": "Parent"
}
```

Success Response (201):

```
{
  "message": "Contact added",
  "contact": {
    "id": 1,
    "user_id": 4,
    "name": "Father",
    "phone": "9876543210",
    "relationship": "Parent",
    "created_at": "2026-02-12T10:00:00.000Z"
  }
}
```

Error Responses:

Code	Condition	Response
400	Missing name/phone	{"error": "Name and phone are required"}
400	Max contacts reached	{"error": "Maximum 3 emergency contacts allowed"}

3.17 Get Emergency Contacts

GET `/api/contacts` **Bearer Token**

Retrieve all emergency contacts for the authenticated user.

Success Response (200):

```
{ "contacts": [ { "id": 1, "name": "Father", "phone": "9876543210", "relationship": "Parent" } ] }
```

3.18 Delete Emergency Contact

DELETE `/api/contacts/:id` **Bearer Token**

Delete an emergency contact by ID.

Success Response (200):

```
{ "message": "Contact deleted" }
```

3.19 Upload Media File

POST `/api/upload` **Bearer Token**

Upload an image or audio file. Max size: 10MB. Accepted formats: JPEG, PNG, GIF, MP3, WAV, OGG, WebM, MP4, M4A. Use `multipart/form-data` with field name `file`.

Request Headers:

Key	Value
Content-Type	multipart/form-data
Authorization	Bearer <token>

Form Data:

Key	Type	Description
file	File	The media file to upload
incident_id	Text (optional)	Link file to an incident

Success Response (201):

```
{
  "message": "File uploaded successfully",
  "file": {
    "id": 1,
    "user_id": 4,
    "filename": "1707...-photo.jpg",
    "original_name": "photo.jpg",
    "mimetype": "image/jpeg",
    "file_size": 245000,
    "created_at": "2026-02-12T10:50:00.000Z"
  }
}
```

3.20 Get User Media Files

GET `/api/upload/media` **Bearer Token**

List all media files uploaded by the authenticated user, with incident association details.

Success Response (200):

```
{ "media": [ { "id": 1, "filename": "...", "original_name": "...", "mimetype": "image/bmp",
"incident_status": "active" } ] }
```

3.21 Delete Media (Password Protected)

POST `/api/upload/delete/:id` **Bearer Token**

Delete a media file. Requires the user's account password for security verification. Deletes both the file from disk and the database record.

Request Body:

```
{ "password": "Test@123" }
```

Success Response (200):

```
{ "message": "Media deleted successfully" }
```

Error Responses:

Code	Condition	Response
400	No password	{"error": "Password is required to delete media"}
403	Wrong password	{"error": "Incorrect password"}
404	Media not found	{"error": "Media not found"}

3.22 Simulate Media Capture

POST `/api/upload/simulate-capture` **Bearer Token**

Simulate camera/audio capture during an emergency. Generates a BMP image (320x240) or WAV audio (2 seconds, 8kHz) and links it to the specified incident.

Request Body:

```
{ "incident_id": 5, "type": "image" }
```

Type can be `"image"` or `"audio"`.

Success Response (201):

```
{
  "message": "image captured (simulated)",
  "file": { "id": 8, "filename": "170...-capture.bmp", "mimetype": "image/bmp", "file_size": 230454
}
}
```

3.23 Legacy Device Endpoints

These endpoints use API key authentication via `X-API-Key` header.

API Key: `vritara-safety-device-key-2024`

Method	Path	Description
POST	/api/sensor-data	Send sensor readings (sound_level, motion_level). Returns emergency flag.
POST	/api/emergency/manual	Legacy manual emergency trigger for devices.
POST	/api/heartbeat	Device heartbeat/alive check.

4. Emergency Flow (PCB Vendor Reference)

4.1 Manual SOS Flow

1. User presses SOS button in app or calls `POST /api/sos/toggle`
2. Backend checks current state. If **normal**, activates emergency (atomic DB transaction)
3. Incident log created with type `"manual"`, status `"active"`, and GPS coordinates
4. User's `emergency_state` set to `"emergency"` and `active_incident_id` stored
5. Simulated SMS sent to all emergency contacts with user's name, location, and emergency message
6. Nearby broadcast: system queries `location_logs` for all users with positions logged in the last hour, calculates Haversine distance, and alerts those within 200 meters
7. App auto-captures evidence: BMP image (320x240) + WAV audio (2 sec, 8kHz), linked to incident
8. User taps SOS again to deactivate: incident status changes to `"resolved"`, state returns to `"normal"`

4.2 Automatic SOS Flow (Hardware Integration)

1. Smartwatch continuously monitors environmental sensors (microphone + accelerometer)
2. When sound > 80dB AND motion > 1.5g, device sends data to `POST /api/sos/automatic`
3. Backend creates incident with type `"automatic"`, stores sensor readings in `sound_level` and `motion_level` fields
4. Full notification flow executes: SMS to contacts + nearby broadcast + incident logging
5. Device continues sending location updates via `POST /api/location`

4.3 Location Storage

Every location update (via `/api/location`) and SOS trigger stores GPS coordinates in `location_logs`. This data is used for:

- Incident location recording
- Nearby user broadcast calculation (Haversine formula, 200m radius)
- Real-time tracking display in the dashboard

4.4 SMS Simulation

SMS alerts are simulated (logged to console and `sms_logs` table). Each SMS includes:

- User's name and emergency type (manual/automatic)
- GPS coordinates (latitude, longitude)
- Urgency message requesting immediate response

> **Production Note:** Replace simulated SMS with Twilio, AWS SNS, or similar SMS gateway for actual SMS delivery.

4.5 Nearby Broadcast (200m Radius)

The system uses the **Haversine formula** to calculate great-circle distance between the SOS user and all other VRITARA users who have logged their location within the last hour. Users within 200 meters receive an emergency broadcast alert stored in the `nearby_broadcasts` table.

4.6 SOS Toggle State

The toggle system uses two fields in the `users` table:

- `emergency_state` : Either `"normal"` or `"emergency"`
- `active_incident_id` : References the current active incident (NULL when normal)

State changes use PostgreSQL transactions (`BEGIN/COMMIT/ROLLBACK`) to ensure atomicity.

5. Database Schema Overview

users

Column	Type	Description
id	SERIAL PK	Auto-increment primary key
username	VARCHAR(100)	Display name
email	VARCHAR(255) UNIQUE	Login email
phone	VARCHAR(20)	Phone number
password	VARCHAR(255)	bcrypt hashed password
otp_code	VARCHAR(6)	Current OTP (nullable)
otp_expires_at	TIMESTAMP	OTP expiry time
reset_token	VARCHAR(100)	Password reset token (nullable)
reset_token_expires_at	TIMESTAMP	Reset token expiry
emergency_state	VARCHAR(20)	'normal' or 'emergency'
active_incident_id	INTEGER	FK to active incident (nullable)
created_at	TIMESTAMP	Account creation time

emergency_contacts

Column	Type	Description
id	SERIAL PK	Auto-increment primary key
user_id	INTEGER FK	References users(id), CASCADE delete
name	VARCHAR(100)	Contact name
phone	VARCHAR(20)	Contact phone number
relationship	VARCHAR(50)	Relationship (default: 'Other')
created_at	TIMESTAMP	Record creation time

incident_logs

Column	Type	Description
id	SERIAL PK	Auto-increment primary key
user_id	INTEGER FK	References users(id)
type	VARCHAR(20)	'manual' or 'automatic'

latitude	DOUBLE PRECISION	GPS latitude
longitude	DOUBLE PRECISION	GPS longitude
message	TEXT	Incident description
sound_level	DOUBLE PRECISION	Detected sound level (dB)
motion_level	DOUBLE PRECISION	Detected motion level (g)
status	VARCHAR(20)	'active' or 'resolved'
created_at	TIMESTAMP	Incident creation time

location_logs

Column	Type	Description
id	SERIAL PK	Auto-increment primary key
user_id	INTEGER FK	References users(id)
latitude	DOUBLE PRECISION	GPS latitude
longitude	DOUBLE PRECISION	GPS longitude
created_at	TIMESTAMP	Location timestamp

sms_logs

Column	Type	Description
id	SERIAL PK	Auto-increment primary key
incident_id	INTEGER FK	References incident_logs(id)
user_id	INTEGER FK	References users(id)
contact_name	VARCHAR(100)	Recipient contact name
contact_phone	VARCHAR(20)	Recipient phone number
message	TEXT	SMS message content
status	VARCHAR(20)	'sent' or 'failed'
created_at	TIMESTAMP	SMS timestamp

nearby_broadcasts

Column	Type	Description
id	SERIAL PK	Auto-increment primary key
incident_id	INTEGER FK	References incident_logs(id)
broadcaster_id	INTEGER FK	User who triggered SOS
receiver_id	INTEGER FK	Nearby user who received alert

distance_meters	DOUBLE PRECISION	Distance between users
message	TEXT	Broadcast message
status	VARCHAR(20)	'sent'
created_at	TIMESTAMP	Broadcast timestamp

media_storage

Column	Type	Description
id	SERIAL PK	Auto-increment primary key
user_id	INTEGER FK	References users(id)
incident_id	INTEGER FK	References incident_logs(id), SET NULL on delete
filename	VARCHAR(255)	Server filename
original_name	VARCHAR(255)	Original upload filename
mimetype	VARCHAR(100)	MIME type (image/bmp, audio/wav, etc.)
file_size	INTEGER	File size in bytes
file_path	VARCHAR(500)	Server file path
created_at	TIMESTAMP	Upload timestamp

Table Relationships

- `users` → `emergency_contacts` (one-to-many, max 3)
- `users` → `incident_logs` (one-to-many)
- `users` → `location_logs` (one-to-many)
- `incident_logs` → `sms_logs` (one-to-many)
- `incident_logs` → `nearby_broadcasts` (one-to-many)
- `incident_logs` → `media_storage` (one-to-many)
- `users` → `media_storage` (one-to-many)

6. Hardware Integration Notes

6.1 Smartwatch Integration

The VRITARA smartwatch device integrates with the backend API to provide automatic emergency detection and response. The device communicates with the following endpoints:

Action	Endpoint	Frequency
Send sensor data	POST /api/sos/automatic	On threshold breach
Update location	POST /api/location	Every 10-30 seconds
Health check	POST /api/heartbeat	Every 60 seconds
Manual SOS button	POST /api/sos/toggle	On user press

6.2 Device Data Payload

The smartwatch sends the following data during automatic SOS detection:

```
{
  "latitude": 12.9716,      // GPS latitude (double precision)
  "longitude": 77.5946,     // GPS longitude (double precision)
  "sound_level": 85,        // Ambient sound in decibels (dB)
  "motion_level": 3.5,      // Accelerometer reading in g-force
  "media_ids": []            // Optional: IDs of pre-uploaded media files
}
```

6.3 Authentication for Device

The smartwatch must authenticate using JWT tokens. Recommended flow:

1. Device performs initial login via `POST /api/login` using stored credentials
2. JWT token stored in device memory (valid for 24 hours)
3. Token refreshed by re-authenticating before expiry
4. All API calls include `Authorization: Bearer <token>` header

6.4 Emergency Detection Thresholds

Sensor	Threshold	Unit	Description
Sound Level	> 80	dB	Ambient noise indicating distress (scream, impact)
Motion Level	> 1.5	g	Sudden acceleration indicating fall or assault

Important: Both thresholds must be exceeded simultaneously for automatic SOS to trigger. The device firmware should implement debouncing to avoid false positives.

6.5 DNA Mechanism

The DNA sample collection mechanism is handled entirely in the wearable hardware. This mechanism is not managed by the backend API. The smartwatch activates the DNA collection module independently during emergency events as part of the physical evidence gathering process.

6.6 Network Requirements

- **Protocol:** HTTPS (TLS 1.2+)
- **Content-Type:** application/json
- **Timeout:** Recommended 10-second timeout per request
- **Retry:** Implement exponential backoff for failed requests (3 retries max)
- **Offline:** Queue SOS events locally and send when connectivity is restored